CRYPTO HAWKING · ETHDENVER 2025 WINNER

# Smart Contract Security Audit

## SatoHook

0x0000f07d2B5F1Ddf3244b8780F972f306EFd2888

Ethereum

Compiler: v0.8.26+commit.8a97fa7a

Date: October 12, 2026

OVERALL RISK RATING AI-assisted audit. Not a substitute for manual review.

**Medium**

# Executive Summary

Independent security review of publicly verified source, not commissioned by the project. SatoHook is a Uniswap v4 hook that replaces the AMM entirely: liquidity additions revert forever, and every swap is served by the hook itself through BeforeSwapDelta, with buys minting SATO along an exponential bonding curve and sells burning it against ETH the hook holds. Two things stand out positively. The contract has no owner, no admin functions and no upgrade path, so there is no privileged party who can redirect funds or change the rules, which is rarer than it should be. The curve mathematics is also correct: the inverse used for sells is the exact analytic inverse of the forward used for buys, and the fee accounting keeps the contract solvent through a full buy and sell cycle. The findings concentrate on the identity and state handling around that core. The caller supplies the swapper address itself in hookData, so the anti sandwich cooldown is bypassed by passing a fresh address, and the same input fed the genesis entropy multiplier that decided the initial distribution. Separately, the curve keeps two linked counters that can silently fall out of step in an edge case, and the sell solvency check measures total balance rather than the reserve that actually backs the curve.

# Findings (8)

F-001

Medium

## Swapper identity is supplied by the caller, so the sell cooldown is bypassed

src/SatoHook.sol:262–263

DESCRIPTION

beforeSwap reads the trader's identity straight out of hookData with abi.decode(hookData, (address)), and that value is what the cooldown and the entropy multiplier key on. Nothing ties it to msg.sender, to tx.origin, or to the account that actually receives the output. A caller buys while passing address A, which sets lastBuyBlock[A], then sells in the same block while passing an unused address B, where lastBuyBlock[B] is zero and the cooldown check is skipped entirely.

IMPACT

The only anti sandwich control in the contract can be stepped around with a single extra word of call-data. Buying and selling atomically in one transaction becomes possible again, which is precisely what COOLDOWN_BLOCKS was written to prevent. Fees make a plain round trip unprofitable today, so this is a defeated control rather than a direct loss, but any future change that makes a round trip profitable would be exploitable immediately.

RECOMMENDATION

Derive the identity from the call itself rather than from calldata. The v4 PoolManager passes the original caller as the first argument of beforeSwap, so use that parameter instead of decoding hookData, or require that the decoded address matches it. If hookData must stay in the interface for routing reasons, treat it as a hint and never as an authorisation.

```
if (hookData.length < 32) revert MissingSwapperInHookData();
address swapper = abi.decode(hookData, (address));

// later, in _executeSell:
uint256 lastBlock = lastBuyBlock[swapper];
if (lastBlock != 0 && (block.number - lastBlock) < COOLDOWN_BLOCKS) revert Cooldown-
Active();
```

F-002

Medium

## The genesis entropy multiplier was fully predictable and grindable

src/SatoHook.sol:434–435

### DESCRIPTION

During the first ENTROPY_BLOCKS blocks, each buy was scaled by a multiplier between 0.9x and 1.1x derived from keccak256(blockhash(block.number - 1), swapper, ethIn). All three inputs are known inside the transaction: the previous block hash is readable on-chain, the swapper address comes from caller controlled hookData per F-001, and ethIn is chosen by the buyer. A contract could therefore loop over candidate address and amount pairs within a single transaction, compute the resulting multiplier for each, and only execute the buy once it found one paying 1.1x.

### IMPACT

The window has long since closed, so this is not a live exploit, and it cannot be fixed retroactively. It does mean the initial distribution was not the lottery it appears to be: a prepared participant could take the maximum bonus on every buy while ordinary buyers received whatever the hash gave them. Because bonus tokens raise actual supply without raising totalMintedFair, the cost of those bonuses is spread across the per token sell value of every holder who came after.

### RECOMMENDATION

No action is possible on the deployed contract, and that is worth stating plainly rather than leaving it implied. For any future curve: never derive a payout multiplier from values the caller controls in the same transaction. Use a commit and reveal over two blocks, or a verifiable randomness source, and keep the identity used for bonuses out of calldata.

```
function _applyEntropy(uint256 fairSato, address swapper, uint256 ethIn) internal view
returns (uint256) {
if (block.number >= GENESIS_BLOCK + ENTROPY_BLOCKS) return fairSato;
bytes32 h = keccak256(abi.encodePacked(blockhash(block.number - 1), swapper,
ethIn));
uint256 mul = 9000 + (uint256(h) % 2001); // 9000..11000
return (fairSato * mul) / 10000;
}
```

F-003

Medium

## Curve position is silently clamped instead of reverting, which can break the invariant

src/SatoHook.sol:419–420

### DESCRIPTION

The contract maintains two counters that must describe the same point on the curve: ethCum, the cumulative ETH in, and totalMintedFair, the supply that maps onto it. On a sell, totalMintedFair is reduced by exactly satoFairIn, but ethCum is reduced by ethRaw only when ethRaw is small enough, and is otherwise set to zero. The two legs are not symmetric, so in the branch where the clamp fires the pair stops describing the same curve position and there is no event or revert to signal it.

### IMPACT

After a clamp, ethCum reads zero while totalMintedFair is still positive. Subsequent buys would then be priced from the very start of the curve, which is the cheapest part, while sells continue to be priced from the much higher totalMintedFair position. The gap is a standing arbitrage against the hook's own reserves. Reaching this state requires accumulated rounding drift in the unfavourable direction, and the UD60x18 helpers truncate

in the protocol's favour, so it is unlikely rather than impossible, which is exactly the kind of assumption worth making explicit.

RECOMMENDATION

Revert instead of clamping. If ethRaw exceeds ethCum the accounting has already diverged, and failing the transaction preserves the invariant while making the condition visible. Alternatively derive one counter from the other, for example recomputing ethCum as Curve.ethAt(totalMintedFair) after each trade, so a single source of truth cannot drift.

```
if (ethRaw <= ethCum) {
ethCum -= ethRaw;
} else {
ethCum = 0;
}
totalMintedFair -= satoFairIn;
```

F-004

Medium

## Sell solvency is checked against total balance, including the accrued fee reserve

src/SatoHook.sol:208–209

DESCRIPTION

The hook separates two pools of ETH conceptually: the curve reserve that backs sells, and feesAccrued, the 0.3% skimmed from both directions. curveReserveEth() reports the difference, which is what external systems read as the backing. The sell path, however, compares the payout against address(this).balance, the whole balance, rather than against the curve reserve. The accounting therefore allows a sell to be paid partly out of fee money.

IMPACT

Once total balance drops below feesAccrued, curveReserveEth() reverts on underflow rather than returning zero, since it is a plain subtraction under checked arithmetic. That breaks every integrator reading it, including the DefiLlama adapter that uses this exact function to report TVL. The protocol does not lose funds, because fees are never withdrawn, but the publicly reported reserve becomes unreadable at precisely the moment people would most want to read it.

RECOMMENDATION

Check the payout against curveReserveEth() rather than the raw balance, so the reserve that backs sells is the one that gates them. Also make curveReserveEth() saturating rather than reverting, returning zero when fees exceed the balance, so that a view function used by third parties can never become a denial of service for them.

```
function curveReserveEth() external view returns (uint256) {
return address(this).balance - feesAccrued;
}

// in _executeSell:
if (address(this).balance < ethOut) revert InsufficientEthReserves();
```

F-005

Low

## Accrued fees have no withdrawal path and accumulate permanently

src/SatoHook.sol:112–369

DESCRIPTION

feesAccrued grows on every buy and every sell and is never reduced anywhere in the contract. There is no withdraw function, no owner, and no recipient address, which follows from the deliberate decision to ship with no privileged roles at all. The effect is that the fee is not revenue in any usable sense, it is ETH that sits in the contract and is excluded from the reported curve reserve.

IMPACT

Whether this is a bug or the intent is impossible to tell from the code alone, and that ambiguity is the finding. If fees were meant to fund the project, they are unreachable. If they were meant to deepen the backing, then excluding them from curveReserveEth() understates the real backing to anyone reading it.

RECOMMENDATION

State the intent in the documentation and make the code agree with it. If the fees are meant to strengthen the floor, stop tracking them separately and let them count toward the reserve. If they were meant to be claimable, that needs a recipient, which means accepting a privileged role the design currently avoids on purpose.

```
uint256 public feesAccrued;

// incremented in both paths, never decremented:
feesAccrued += fee;
```

F-006

Low

## Sell reverts with a division by zero if supply ever reaches zero

src/SatoHook.sol:400–401

DESCRIPTION

_executeSell normalises the seller's tokens into fair curve units by dividing by SATO_TOKEN.totalSupply(). The call is unguarded, so a totalSupply of zero produces a panic rather than a clear error. In practice a seller must hold tokens for supply to be non zero, which makes the state hard to reach, but the ordering means the check depends on an external contract rather than on local state.

IMPACT

Limited to a confusing failure mode: a panic instead of a named error makes the revert harder to diagnose for integrators, and it ties the hook's liveness to the token's accounting.

RECOMMENDATION

Guard the denominator explicitly and revert with a named error when supply is zero. The pattern also makes the dependency on the external token contract visible to anyone reading the function.

```
uint256 actualSupply = SATO_TOKEN.totalSupply();
uint256 satoFairIn = (satoIn * totalMintedFair) / actualSupply;
```

F-007

Informational

## Open receive lets anyone inflate the reported reserve

src/SatoHook.sol:445–undefined

DESCRIPTION

receive() is payable and unconditional, so any account can send ETH directly to the hook. Donated ETH raises address(this).balance and therefore curveReserveEth(), while ethCum and totalMintedFair are untouched, so the curve prices nothing differently.

IMPACT

Donations strengthen the real backing, which is harmless and arguably good. The side effect is that a reported reserve figure can be moved by anyone, so dashboards and TVL feeds built on curveReserveEth() reflect contributions that are not part of the curve.

RECOMMENDATION

Keep accepting ETH, since the PoolManager settlement path needs it, but emit an event when the sender is not the PoolManager, so unexpected inflows are visible to anyone monitoring the contract.

```
receive() external payable {}
```

F-008

Informational

## No slippage bound inside the hook, routers carry that responsibility

src/SatoHook.sol:253–254

DESCRIPTION

The hook prices a swap from live curve state and returns a delta without any minimum output or maximum input parameter. A trader's protection depends entirely on the router they go through, and the implementation note confirms routers must also pre-settle the input currency before calling swap, which is an unusual integration requirement.

IMPACT

A router that omits a minimum output check exposes its users to whatever the curve returns at execution time, including the result of another trade landing first in the same block. The hook cannot defend against this on their behalf.

RECOMMENDATION

Document the required router behaviour prominently next to the published address, including both the pre-settlement step and the expectation that the router enforces slippage. Publishing a reference router implementation is the most effective way to make correct integration the default.

```
function beforeSwap(address, PoolKey calldata key, SwapParams calldata params, bytes
calldata hookData)
external onlyPoolManager
returns (bytes4 selector, BeforeSwapDelta delta, uint24 lpFeeOverride)
```

# Gas Optimizations

### Cache totalMintedFair and ethCum in the sell path

src/SatoHook.sol:401–405

_executeSell reads totalMintedFair from storage for the normalisation, again for the burnFor call, and writes it back at the end, while ethCum is read twice in the clamp. Loading both into local variables once and writing back a single time removes several warm storage reads from the hot path.

### Build the manifesto string without the intermediate 256 byte copy

src/SatoHook.sol:186–187

philosophy() allocates a 256 byte buffer, fills it byte by byte through a nested loop, then allocates a second trimmed buffer and copies again. Writing the words directly into memory and setting the length once avoids the second allocation and the per byte loop entirely. It is a view function, so this only matters for off-chain callers, but the saving is large.

### Mark the unused IHooks entrypoints as reverting rather than returning

src/SatoHook.sol:276–281

Six interface functions exist only to satisfy IHooks and can never be invoked, because the permission bits encoded in the deployed address exclude them. Reverting instead of returning a selector documents that intent in the bytecode and shrinks the deployed size slightly.

# Appendix · Metadata

| | |
|---|---|
| Contract | SatoHook |
| Address | 0x0000f07d2B5F1Ddf3244b8780F972f306EFd2888 |
| Chain | Ethereum |
| Compiler | v0.8.26+commit.8a97fa7a |
| Audit ID | independent-sato |
| Date | 10/12/2026 |

Files analyzed

```
[user] src/SatoHook.sol
[user] src/lib/Curve.sol
[user] src/SatoToken.sol
```

MANUAL AUDIT UPGRADE

# Need a deeper audit?

# $5,000

Pay in ETH, SOL, or USDT. Detailed manual review by Crypto Hawking, ETHDenver 2025 Winner. Comprehensive findings, remediation steps, and post-fix re-review. Report delivered in 3 business days.

**Request Manual Audit ’**

https://cryptohawking.com/audit/manual?ref=independent-sato

## Need it built, not just audited?

I also build production smart contracts end to end, token launches, DeFi protocols, staking, and full DEX stacks (v2, v3 concentrated liquidity, and beyond), plus fractional CTO engagements for Web3 teams. Fixes for the findings in this report can be scoped in the same engagement.

https://cryptohawking.com/services

WhatsApp (preferred): +92 322 4274236, https://wa.me/923224274236